

Cineloomb: System Design, Business Workflows & Backend Architecture

Enterprise Architecture Specification: ASP.NET Core C# Microservices, Relational SQL Engine, Concurrency & API Design

Project: Cineloomb Cinema Platform	Authors: By Anshi, Mehak and Sachin
Backend Engine: ASP.NET Core (.NET 8 C# Web API)	Database Engine: Microsoft SQL Server / PostgreSQL (ACID 3NF)
Concurrency Strategy: Distributed Lock + Optimistic RowVersion	Architecture Pattern: Clean Architecture & Domain-Driven Design

Executive System Design Summary

Cineloomb is engineered as a resilient, high-throughput movie ticketing platform capable of handling intense traffic surges during blockbuster ticket releases. The system addresses critical distributed systems challenges including zero-collision seat reservations, atomic multi-seat transactions, dynamic surcharge calculations, conflict-free screening scheduling, idempotent payment settlement, and automated refund auditing. This document details the end-to-end system design, business workflows, SQL database schema, and C# .NET backend architecture.

Core System Capabilities

- Real-Time Seat Locking: 5-minute temporary hold with distributed TTL and database row isolation.
- Dynamic Pricing Engine: Time-of-day, weekend multiplier, screen format fees, and promo discounts.
- Collision-Free Scheduling: Mathematical screen timeline collision validator with turnaround buffers.
- Automated Tiered Cancellation: Time-to-showtime refund calculation (80% / 50% / 0%) with instant seat recovery.
- Clean Architecture Backend: Decoupled domain, application, infrastructure, and RESTful API layers in C# .NET.

1. High-Level System Architecture & Topology

The Cineloom platform adopts a Clean Layered Architecture with decoupled responsibilities across the application boundary:



2. End-to-End Business & Data Workflows

Flow 1: Movie Discovery & Multi-Criteria Search

1. The client submits a query with optional filters (Genre, Language, Format, City, Status).
2. The API checks the Redis catalog cache. On cache hit, results return in sub-10ms.
3. On cache miss, Dapper executes a parameterized SQL query filtering active movies, caches the result set for 15 minutes, and returns the response.

Flow 2: Seat Availability, Concurrency & 5-Minute Locking

1. The customer selects a showtime. The backend retrieves the seat layout with current states (Available, Locked, Booked).
2. The customer selects up to 8 seats and clicks 'Lock Seats'.
3. The backend opens a database transaction with Row-Level Isolation and checks if any requested seat is in 'BOOKED' or 'LOCKED' status.
4. If available, seats are updated to 'LOCKED', assigned the customer's UserId, and given a LockExpiresAt timestamp (Current UTC + 300 seconds).
5. A background worker periodically sweeps and resets any expired locks whose LockExpiresAt < Current UTC.

Flow 3: Dynamic Price Computation & Offer Validation

1. Base seat tier rates are resolved: Regular (\$12.00), Premium (\$16.50), VIP Recliner (\$22.00).
2. Format surcharge is added (IMAX 3D +\$4.00, 4DX +\$5.00, Dolby +\$3.00, 3D +\$2.00).
3. Weekend multiplier is applied (15% surcharge for Saturday and Sunday shows).
4. Promo voucher validation checks active status, expiration date, and minimum booking threshold, applying percentage caps.
5. Convenience fee (\$1.50/seat) and GST/Tax (12%) are added to calculate the immutable grand total.

Flow 4: Payment Authorization & Atomic Booking Confirmation

1. The client sends payment details and the locked seat token.
2. The payment gateway verifies funds and returns a successful transaction ID.
3. Within a single atomic database transaction: seats transition from 'LOCKED' to 'BOOKED', a unique booking reference (e.g. CNL-782910) is generated, a Booking record is committed, and an audit entry is logged.
4. If payment fails, the transaction rolls back and seats are immediately released.

Flow 5: Booking Cancellation & Tiered Refund Processing

1. The customer initiates cancellation from booking history.
2. The backend computes hours remaining until showtime: >2 hours grants 80% refund of base ticket price; 1 to 2 hours grants 50%; <1 hour is non-refundable.
3. The booking status updates to 'REFUNDED', convenience fees are retained, booked seats are freed back to the available inventory, and an audit receipt is recorded.

3. Relational Database Design & SQL Schema

A normalized third-normal-form (3NF) relational database schema enforces strict referential integrity and zero data duplication:

A. Master Master Tables (Users, Theatres & Screens)

```
CREATE TABLE Users (  
    UserId NVARCHAR(50) PRIMARY KEY,  
    FullName NVARCHAR(100) NOT NULL,  
    Email NVARCHAR(120) UNIQUE NOT NULL,  
    PasswordHash NVARCHAR(255) NOT NULL,  
    PhoneNumber NVARCHAR(20),  
    Role NVARCHAR(20) DEFAULT 'CUSTOMER', -- 'CUSTOMER', 'ADMINISTRATOR'  
    City NVARCHAR(50) DEFAULT 'New York',  
    CreatedAt DATETIME2 DEFAULT SYSUTCDATETIME()  
);  
  
CREATE TABLE Theatres (  
    TheatreId NVARCHAR(50) PRIMARY KEY,  
    Name NVARCHAR(150) NOT NULL,  
    City NVARCHAR(50) NOT NULL,  
    LocationAddress NVARCHAR(255) NOT NULL,  
    Rating DECIMAL(2,1) DEFAULT 4.8,  
    IsActive BIT DEFAULT 1  
);  
  
CREATE TABLE Screens (  
    ScreenId NVARCHAR(50) PRIMARY KEY,  
    TheatreId NVARCHAR(50) FOREIGN KEY REFERENCES Theatres(TheatreId) ON DELETE CASCADE,  
    Name NVARCHAR(100) NOT NULL,  
    ScreenFormat NVARCHAR(30) NOT NULL, -- '2D', '3D', 'IMAX 3D', '4DX', 'Dolby Cinema'  
    TotalCapacity INT NOT NULL,  
    IsActive BIT DEFAULT 1  
);
```

B. Catalog & Showtime Scheduling Tables

```

CREATE TABLE Movies (
    MovieId NVARCHAR(50) PRIMARY KEY,
    Title NVARCHAR(200) NOT NULL,
    Genre NVARCHAR(50) NOT NULL,
    Language NVARCHAR(50) NOT NULL,
    DurationMinutes INT NOT NULL,
    Certification NVARCHAR(10) NOT NULL, -- 'U', 'UA13+', 'UA16+', 'A'
    Rating DECIMAL(2,1) DEFAULT 8.5,
    ReleaseDate DATE NOT NULL,
    Status NVARCHAR(20) NOT NULL, -- 'NOW_SHOWING', 'UPCOMING', 'ARCHIVED'
    Director NVARCHAR(100),
    CastMembers NVARCHAR(MAX),
    Description NVARCHAR(MAX),
    PosterUrl NVARCHAR(500),
    BannerUrl NVARCHAR(500),
    IsActive BIT DEFAULT 1
);

CREATE TABLE Shows (
    ShowId NVARCHAR(50) PRIMARY KEY,
    MovieId NVARCHAR(50) FOREIGN KEY REFERENCES Movies(MovieId),
    TheatreId NVARCHAR(50) FOREIGN KEY REFERENCES Theatres(TheatreId),
    ScreenId NVARCHAR(50) FOREIGN KEY REFERENCES Screens(ScreenId),
    ShowDate DATE NOT NULL,
    StartTime NVARCHAR(10) NOT NULL, -- e.g. '07:30 PM'
    EndTime NVARCHAR(10) NOT NULL,
    BasePrice DECIMAL(10,2) NOT NULL,
    IsActive BIT DEFAULT 1
);

-- Seat Concurrency Table with SQL Server RowVersion Token
CREATE TABLE ShowSeats (
    ShowSeatId NVARCHAR(100) PRIMARY KEY,
    ShowId NVARCHAR(50) FOREIGN KEY REFERENCES Shows(ShowId),
    SeatNumber NVARCHAR(10) NOT NULL, -- e.g. 'D4', 'H9'
    CategoryName NVARCHAR(30) NOT NULL, -- 'Regular', 'Premium', 'VIP Recliner'
    SeatStatus NVARCHAR(20) NOT NULL, -- 'AVAILABLE', 'LOCKED', 'BOOKED'
    LockedByUserId NVARCHAR(50) NULL,
    LockExpiresAt DATETIME2 NULL,
    RowVersion ROWVERSION -- Optimistic Concurrency Token
);

```

C. Booking, Transaction & Audit Tables

```
CREATE TABLE Bookings (  
    BookingId NVARCHAR(50) PRIMARY KEY,  
    BookingReference NVARCHAR(20) UNIQUE NOT NULL, -- e.g. 'CNL-782910'  
    UserId NVARCHAR(50) FOREIGN KEY REFERENCES Users(UserId),  
    ShowId NVARCHAR(50) FOREIGN KEY REFERENCES Shows(ShowId),  
    SeatCount INT NOT NULL,  
    Subtotal DECIMAL(10,2) NOT NULL,  
    DiscountAmount DECIMAL(10,2) DEFAULT 0.0,  
    AppliedCouponCode NVARCHAR(30) NULL,  
    ConvenienceFee DECIMAL(10,2) NOT NULL,  
    TaxAmount DECIMAL(10,2) NOT NULL,  
    TotalAmount DECIMAL(10,2) NOT NULL,  
    BookingStatus NVARCHAR(20) NOT NULL, -- 'CONFIRMED', 'CANCELLED', 'REFUNDED'  
    PaymentStatus NVARCHAR(20) NOT NULL, -- 'SUCCESSFUL', 'FAILED', 'REFUNDED'  
    PaymentMethod NVARCHAR(30) NOT NULL, -- 'CARD', 'UPI', 'NET_BANKING'  
    CreatedAt DATETIME2 DEFAULT SYSUTCDATETIME()  
);  
  
CREATE TABLE AuditLogs (  
    LogId NVARCHAR(50) PRIMARY KEY,  
    Timestamp DATETIME2 DEFAULT SYSUTCDATETIME(),  
    ActionType NVARCHAR(50) NOT NULL, -- 'BOOKING_CONFIRMED', 'SEATS_LOCKED', etc.  
    UserId NVARCHAR(50) NULL,  
    Severity NVARCHAR(20) NOT NULL, -- 'INFO', 'WARN', 'ERROR', 'AUDIT'  
    EventPayload NVARCHAR(MAX) NOT NULL  
);
```

4. C# .NET Backend Architecture (ASP.NET Core Web API)

The ASP.NET Core backend implements Domain-Driven Design (DDD) with Dependency Injection:

```
// BookingController.cs – ASP.NET Core REST API Controller
[ApiController]
[Route("api/[controller]")]
[Authorize]
public class BookingController : ControllerBase
{
    private readonly IBookingService _bookingService;
    private readonly ISeatService _seatService;
    private readonly ILogger<BookingController> _logger;

    public BookingController(IBookingService bookingService, ISeatService seatService, ILogger<BookingController> logger)
    {
        _bookingService = bookingService;
        _seatService = seatService;
        _logger = logger;
    }

    [HttpPost("lock-seats")]
    public async Task<IActionResult> LockSeats([FromBody] LockSeatsDto request)
    {
        var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
        var result = await _seatService.LockSeatsAsync(request.ShowId, request.SeatNumbers, userId);
        if (!result.IsSuccess)
            return StatusCode(StatusCodes.Status409Conflict, new { message = result.ErrorMessage });

        return Ok(new { success = true, data = result.Data });
    }

    [HttpPost("confirm")]
    public async Task<IActionResult> ConfirmBooking([FromBody] CreateBookingDto request)
    {
        var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
        var booking = await _bookingService.CreateAndConfirmBookingAsync(userId, request);
        return Ok(new { success = true, booking });
    }

    [HttpPost("{bookingId}/cancel")]
    public async Task<IActionResult> CancelBooking(string bookingId, [FromBody] CancelRequestDto dto)
    {
        var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
        var refundResult = await _bookingService.ProcessCancellationAsync(bookingId, userId, dto.Reason);
        return Ok(refundResult);
    }
}
```

5. Complete REST API Specifications & Contracts

Standardized REST API contracts consumed by the frontend and external integrations:

HTTP Route	Auth	Request Payload	Response Contract
POST /api/auth/login	Public	{ email, password }	{ token, userProfile }
POST /api/auth/register	Public	{ name, email, password, phone, role }	{ token, userProfile }
GET /api/movies	Public	?genre=Sci-Fi&status;=NOW_SHOWING	List
GET /api/shows	Public	?movieId=mov-1&date;=2026-08-16&city;=New York	List
GET /api/seats/layout/{showId}	Public	None	SeatMatrixLayoutDto
POST /api/seats/lock	Bearer JWT	{ showId, seatNumbers: ['D4', 'D5'] }	{ lockExpiresAt, lockedSeats }
POST /api/booking/confirm	Bearer JWT	{ showId, seats, couponCode, paymentDetails }	ConfirmedBookingPassDto
POST /api/booking/{id}/cancel	Bearer JWT	{ reason: 'Schedule change' }	{ refundAmount, refundPercentage, status }

POST /api/admin/shows/schedule	Admin JWT	{ movieId, screenId, date, startTime, price }	{ showId, screenConflict: false }
---------------------------------------	-----------	---	-----------------------------------

6. Concurrency Strategy, Fault Tolerance & Scale

A. Race Condition Prevention Mechanism

To guarantee that two concurrent requests for the same seat cannot both succeed, Cineloom utilizes a two-tier protection strategy:

1. **Distributed In-Memory Lock (Redis)**: An atomic SETNX command acquires an exclusive lock key:

``lock:show:{showId};seat:{seatNumber}`` with a 300-second TTL.

2. **Database RowVersion Optimistic Concurrency**: If two transactions pass the memory lock simultaneously, SQL Server's RowVersion token detects the collision upon commit and throws a `DbUpdateConcurrencyException`, rolling back the loser transaction.

B. Show Scheduling Collision Detection Algorithm

To eliminate screen double-booking, start times and movie durations are mapped to integer minutes on the day timeline, including a mandatory 20-minute turnaround and cleaning buffer. The scheduling service validates:

Overlap = (NewShowStart < ExistingShowEnd) AND (NewShowEnd > ExistingShowStart).

If true, the conflict is returned with the name and timeslot of the blocking show, and scheduling is prohibited.

C. Tiered Cancellation & Instant Refund Engine

Calculates time remaining until showtime:

- More than 2 hours before show: 80% refund of base ticket price.
- Between 1 and 2 hours before show: 50% refund.
- Less than 1 hour before show: 0% refund (non-refundable).

Convenience fees are non-refundable. Released seats are immediately restored to the active inventory.

7. Project Presentation Summary

Presentation Sign-Off Note

Cineloom represents an enterprise full-stack system design bridging intuitive customer interactions with robust .NET C# backend microservices and an ACID-compliant SQL schema. The architecture guarantees zero seat collisions, dynamic pricing precision, conflict-free scheduling, and complete auditability under peak cinema booking loads.

Presented by: Anshi, Mehak and Sachin

Cineloom Cinema Networks • Master System Design & Technical Specification